

Problem Description:

You are given an array A of Q integers and Q queries. In each query, you are given an integer i ($1 \leq i \leq N$).

Your task is to find the minimum index greater than i ($1 \leq i \leq N$) such that:

1. Sum of digits of A_i is greater than the sum of digits of A_j
2. $A_i < A_j$

Question No:3

If there is no answer, then print **-1**.

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq A_i \leq 10^9$$

$$1 \leq Q_i \leq N$$

Input format

- The first line contains two numbers N and Q .
- The next line contains N numbers.
- Next Q lines contain Q queries.

Output format

Print the answer as described in the problem

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5 5
62 70 28 62 92
1
5
3
4
2

EXPECTED OUTPUT

2 -1 4 -1 -1

Test Case 2

INPUT (STDIN)

6 6
12 71 21 16 91
11
15
31
14
21

EXPECTED OUTPUT

-1 -1 -1 -1 -1 -1

▼ Mandatory Test Cases

Test Case 1

KEYWORD

if(arr[x]<arr[y])

Test Case 2

KEYWORD

if(arr2[x]>arr2[y])

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

10

Test Case 2

TOKEN COUNT

310

Test Case 3

NLOC

57

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct {
5      int value;
6      int sum;
7  } Pair;
8
9  typedef struct {
10     Pair *arr;
11     int *suffixMin;
12     int size;
13 } Node;
14
15 int digitSum(int n)
16 {
17     int sum = 0;
18
19     while (n > 0)
20     {
21         sum += n % 10;
22         n /= 10;
23     }
24
25     return sum;
26 }
27
28 void build(int node, int left, int right, Pair *a, Node *tree)
29 {
30     if (left == right)
31     {
32         tree[node].size = 1;
33     }
```

```

34     tree[node].arr = (Pair *)malloc(sizeof(Pair));
35     tree[node].suffixMin = (int *)malloc(sizeof(int));
36
37     tree[node].arr[0] = a[left];
38     tree[node].suffixMin[0] = a[left].sum;
39
40     return;
41 }
42
43 int mid = (left + right) / 2;
44
45 build(node * 2, left, mid, a, tree);
46 build(node * 2 + 1, mid + 1, right, a, tree);
47
48 int n1 = tree[node * 2].size;
49 int n2 = tree[node * 2 + 1].size;
50
51 tree[node].size = n1 + n2;
52
53 tree[node].arr =
54     (Pair *)malloc(tree[node].size * sizeof(Pair));
55
56 int i = 0;
57 int j = 0;
58 int k = 0;
59
60 while (i < n1 && j < n2)
61 {
62     if (tree[node * 2].arr[i].value <=
63         tree[node * 2 + 1].arr[j].value)
64     {

```

```

65         tree[node].arr[k++] =
66             tree[node * 2].arr[i++];
67     }
68     else
69     {
70         tree[node].arr[k++] =
71             tree[node * 2 + 1].arr[j++];
72     }
73 }
74
75 while (i < n1)
76 {
77     tree[node].arr[k++] =
78         tree[node * 2].arr[i++];
79 }
80
81 while (j < n2)
82 {
83     tree[node].arr[k++] =
84         tree[node * 2 + 1].arr[j++];
85 }
86
87 tree[node].suffixMin =
88     (int *)malloc(tree[node].size * sizeof(int));
89
90 tree[node].suffixMin[tree[node].size - 1] =
91     tree[node].arr[tree[node].size - 1].sum;
92
93 for (i = tree[node].size - 2; i >= 0; i--)
94 {

```



```

95     tree[node].suffixMin[i] =
96         tree[node].arr[i].sum;
97
98     if (tree[node].suffixMin[i + 1] <
99         tree[node].suffixMin[i])
100     {
101         tree[node].suffixMin[i] =
102             tree[node].suffixMin[i + 1];
103     }
104 }
105 }
106
107 /* Find first value strictly greater than x */
108 int upperBound(Pair *arr, int size, int x)
109 {
110     int left = 0;
111     int right = size;
112
113     while (left < right)
114     {
115         int mid = (left + right) / 2;
116
117         if (arr[mid].value <= x)
118             left = mid + 1;
119         else
120             right = mid;
121     }
122
123     return left;
124 }
125
126 /*
127     Checks whether this segment contains an element

```

```
128     satisfying:
129
130     value > targetValue
131     digitSum < targetSum
132     */
133 int exists(Node *tree, int node, int targetValue, int targetSum)
134 {
135     int pos;
136
137     pos = upperBound(
138         tree[node].arr,
139         tree[node].size,
140         targetValue
141     );
142
143     if (pos == tree[node].size)
144         return 0;
145
146     if (tree[node].suffixMin[pos] < targetSum)
147         return 1;
148
149     return 0;
150 }
151
152 /* Find the first valid index */
153 int findFirst(
154     Node *tree,
155     int node,
156     int left,
157     int right,
158     int start,
159     int targetValue,
160     int targetSum
```



```
161 )
162 {
163     int mid;
164     int answer;
165
166     if (right < start)
167         return -1;
168
169     if (left >= start)
170     {
171         if (!exists(tree, node, targetValue, targetSum))
172             return -1;
173     }
174
175     if (left == right)
176         return left;
177
178     mid = (left + right) / 2;
179
180     answer = findFirst(
181         tree,
182         node * 2,
183         left,
184         mid,
185         start,
186         targetValue,
187         targetSum
188     );
189
190     if (answer != -1)
191         return answer;
192
193     return findFirst(
```

```

194         tree,
195         node * 2 + 1,
196         mid + 1,
197         right,
198         start,
199         targetValue,
200         targetSum
201     );
202 }
203
204 int main()
205 {
206     int N, Q;
207     int i, query;
208     Pair *a;
209     Node *tree;
210
211     scanf("%d %d", &N, &Q);
212
213     a = (Pair *)malloc(N * sizeof(Pair));
214
215     for (i = 0; i < N; i++)
216     {
217         scanf("%d", &a[i].value);
218         a[i].sum = digitSum(a[i].value);
219     }
220
221     tree = (Node *)calloc(4 * N, sizeof(Node));
222
223     build(1, 0, N - 1, a, tree);
224 }

```

```
224
225     for (i = 0; i < Q; i++)
226     {
227         scanf("%d", &query);
228
229         /* Convert 1-based index to 0-based */
230         query--;
231
232         if (query == N - 1)
233         {
234             printf("-1 ");
235         }
236         else
237         {
238             int answer;
239
240             answer = findFirst(
241                 tree,
242                 1,
243                 0,
244                 N - 1,
245                 query + 1,
246                 a[query].value,
247                 a[query].sum
248             );
249
250             if (answer == -1)
251                 printf("-1 ");
252             else
253                 printf("%d ", answer + 1);
254         }
255     }
```

```
256
257     return 0;
258 }
```

Output:
Test Case-1:

Output				
5	5			
62	70	28	62	92
1				
5				
3				
4				
2				
2	-1	4	-1	-1

Test Case-2:

	6 6	
	12 71 21 16 91	
	11	
	15	
	-1	
	31	
	-1	
	14	
	-1	
	21	
	-1	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236